

HDOC - Day 4 Activity

Question: Find the product of two integers without using the multiplication operator (*), using repeated addition with the minimum possible number of additions.

1. Understanding the Problem

- Input: Two integers A and B.
- Output: Product of A and B.
- Restriction: Do not use the multiplication operator (*).
- Method: Use repeated addition.
- Minimum additions: Add the number with the larger absolute value only |smaller number| times.
- Special cases: Correctly handle all sign combinations and zero.

2. Mistakes in the Given/Incorrect Algorithm

- Repeating addition according to the first input only can use more additions than necessary. Example: 12 and 3 should require 3 additions, not 12.
- Using a negative number directly as the loop count can make the loop incorrect or prevent it from running.
- Not separating magnitude from sign gives wrong results for positive x negative, negative x positive, and negative x negative.
- Zero must be handled separately so that the product is 0 with 0 additions.
- The final sign must be negative only when exactly one input is negative.
- The algorithm must count additions using $\min(|A|, |B|)$, not simply A or B.

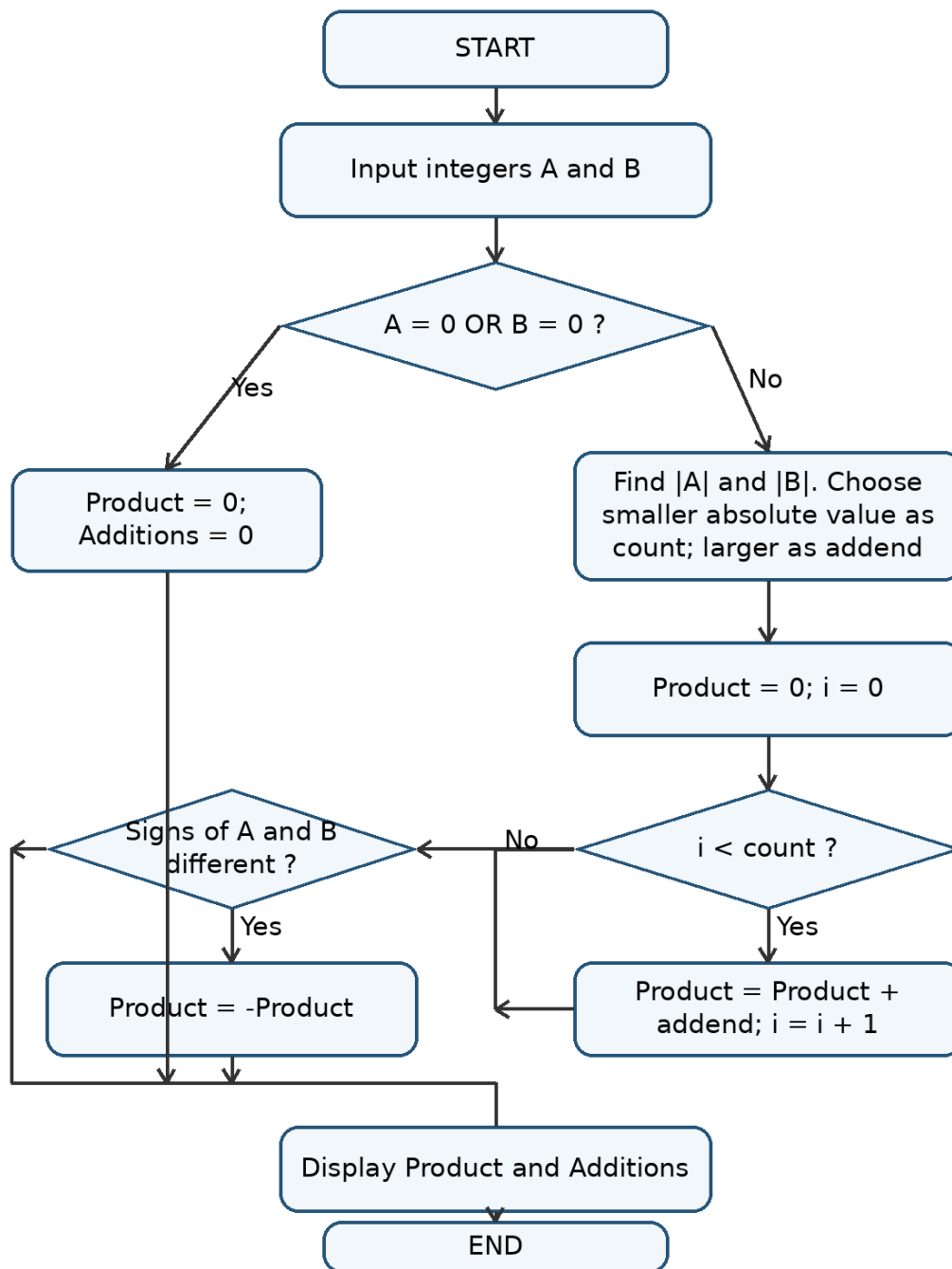
3. Corrected Algorithm

1. Start.
2. Read integers A and B.
3. If $A = 0$ or $B = 0$, set Product = 0 and Additions = 0; display the result and stop.
4. Find $\text{absA} = |A|$ and $\text{absB} = |B|$.
5. If $\text{absA} \leq \text{absB}$, set count = absA and addend = absB; otherwise set count = absB and addend = absA.
6. Set Product = 0 and $i = 0$.
7. Repeat while $i < \text{count}$: Product = Product + addend; $i = i + 1$.
8. Set Additions = count.
9. If A and B have different signs, change Product to -Product.
10. Display Product and Additions.
11. Stop.

4. Mistakes in the Given/Incorrect Flowchart

- It must include a zero check before entering the repeated-addition loop.
- It must compare $|A|$ and $|B|$ so that the smaller absolute value controls the number of additions.
- The loop must use non-negative magnitudes; otherwise negative inputs can break the loop logic.
- It must include a sign decision after repeated addition to make the result negative when the input signs differ.
- All decision branches must reconnect correctly to output/stop, including the zero branch.

5. Corrected Flowchart



6. Verification Using the Given Test Cases

Test	Input (A, B)	Expected	Actual	Additions	Result
1	12, 3	Product: 36	Product: 36	3	Pass
2	3, 12	Product: 36	Product: 36	3	Pass
3	-6, 5	Product: -30	Product: -30	5	Pass
4	6, -5	Product: -30	Product: -30	5	Pass
5	-9, -4	Product: 36	Product: 36	4	Pass
6	-4, -9	Product: 36	Product: 36	4	Pass
7	0, 8	Product: 0	Product: 0	0	Pass
8	8, 0	Product: 0	Product: 0	0	Pass
9	0, -7	Product: 0	Product: 0	0	Pass
10	-7, 0	Product: 0	Product: 0	0	Pass
11	0, 0	Product: 0	Product: 0	0	Pass
12	1, -7	Product: -7	Product: -7	1	Pass
13	-1, -9	Product: 9	Product: 9	1	Pass

7. C Program

```
#include <stdio.h>

int main()
{
    long long a, b, absA, absB;
    long long addend, count, product = 0, i;

    printf("Enter two integers: ");
    scanf("%lld %lld", &a, &b);

    if (a == 0 || b == 0)
    {
        printf("Product: 0\n");
        printf("Number of additions: 0\n");
        return 0;
    }

    absA = (a < 0) ? -a : a;
    absB = (b < 0) ? -b : b;

    if (absA <= absB)
    {
        count = absA;
        addend = absB;
    }
    else
    {
        count = absB;
        addend = absA;
    }

    for (i = 0; i < count; i++)
        product = product + addend;
```

```
if ((a < 0 && b > 0) || (a > 0 && b < 0))
    product = -product;

printf("Product: %lld\n", product);
printf("Number of additions: %lld\n", count);

return 0;
}
```

8. Kali Linux Commands for Compilation and Execution

```
(kali@kali)-[~]
└─$ nano day4.c
# Paste the C program, then press Ctrl+O, Enter, Ctrl+X
(kali@kali)-[~]
└─$ gcc day4.c -o day4
(kali@kali)-[~]
└─$ ./day4
```

9. Sample Kali Linux Executions

Test Case 1:

```
(kali@kali)-[~]
└─$ ./day4
Enter two integers: 12 3
Product: 36
Number of additions: 3
```

Test Case 5:

```
(kali@kali)-[~]
└─$ ./day4
Enter two integers: -9 -4
Product: 36
Number of additions: 4
```

Test Case 12:

```
(kali@kali)-[~]
└─$ ./day4
Enter two integers: 1 -7
Product: -7
Number of additions: 1
```

Zero Case:

```
(kali@kali)-[~]
```

└─\$./day4

Enter two integers: 0 -7

Product: 0

Number of additions: 0

10. Conclusion

The corrected logic works for positive, negative, and zero inputs. It avoids the multiplication operator and uses exactly $\min(|A|, |B|)$ repeated additions for non-zero inputs, which matches the minimum-addition values specified in the Day-4 question sheet.